Exercise 1 - Merge Sort

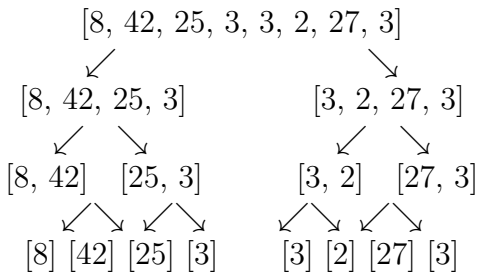
2. Complexity Analysis

The merge sort implementation has $O(n \log n)$ worst-case complexity because it recursively splits the array into halves (creating $\log n$ recursive levels) and at each level, the merge() function processes all n elements once when combining sorted subarrays. Since there are $\log n$ levels and each level performs $O(n)$ work, the total computational cost is $O(n \log n)$. This is true regardless of the ordering of elements because the algorithm will always fully split and merge every subarray (meaning it is the worst-case, average-case, and best-case complexity for this implementation).

3. Manual Application for [8, 42, 25, 3, 3, 2, 27, 3]

Splitting:

First, split the array in half repeatedly until we have single elements:



Merging Phase:

1: Merge [8] and [42] → compare 8 and 42 → [8, 42]

2: Merge [25] and [3] → compare 25 and 3 → [3, 25]

3: Merge [3] and [2] → compare 3 and 2 → [2, 3]

4: Merge [27] and [3] → compare 27 and 3 → [3, 27]

Now we have [8, 42, 3, 25, 2, 3, 3, 27]

5: Merge [8, 42] and [3, 25]

- Compare 8 vs 3 → 3 is smaller → take 3
- Compare 8 vs 25 → 8 is smaller → take 8
- Compare 42 vs 25 → 25 is smaller → take 25
- Take remaining 42

Result: [3, 8, 25, 42]

Step 6: Merge [2, 3] and [3, 27]

- Compare 2 vs 3 → 2 is smaller → take 2

- Compare 3 vs 3 → take left 3
- Compare right 3 vs 27 → take 3
- Take remaining 27

Result: [2, 3, 3, 27]

After steps 5 and 6: [3, 8, 25, 42, 2, 3, 3, 27]

Step 7: Final merging of [3, 8, 25, 42] and [2, 3, 3, 27]

- Compare 3 vs 2 → 2 is smaller → take 2
- Compare 3 vs 3 → take left 3
- Compare 8 vs 3 → take right 3
- Compare 8 vs 3 → take right 3
- Compare 8 vs 27 → 8 is smaller → take 8
- Compare 25 vs 27 → 25 is smaller → take 25
- Compare 42 vs 27 → 27 is smaller → take 27
- Take remaining 42

Final sorted array: [2, 3, 3, 3, 8, 25, 27, 42]

4. Does This Match the Complexity Analysis?

For an array of size $n = 8$:

- Merge sort has $\log_2 8 = 3$ levels of recursion
- All elements are processed at each level
- Upper bound = $n \log_2 n = 8 \times 3 = 24$ comparisons
- Manual example used 17 comparisons, which falls within the expected range.

The algorithm processes each element once per level, giving $n \log n$ total operations.